

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date: 2/4/2026	Enrolment No: 92301733059

Aim: To recommend an item to the user using collaborative-based filtering approach

IDE: Google Colab

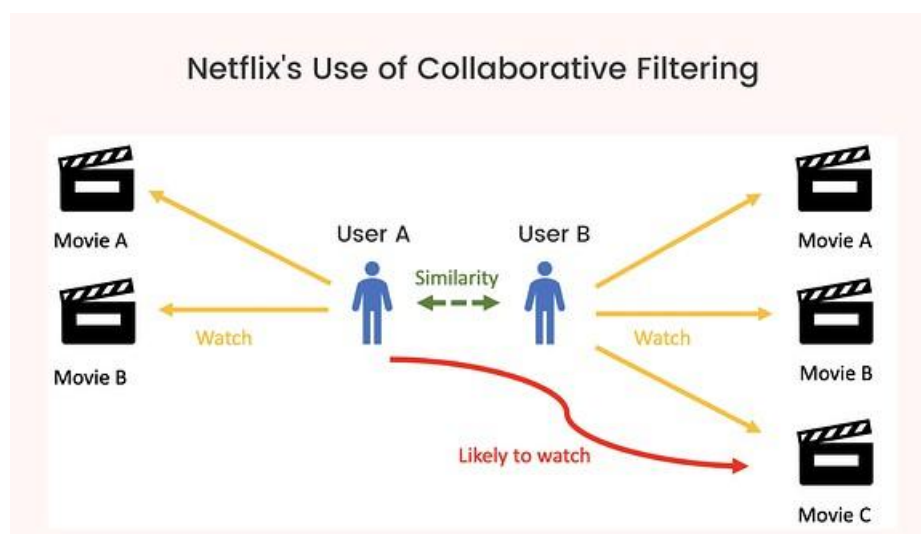
Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

In Collaborative Filtering, we tend to find similar users and recommend what similar users like. In this type of recommendation system, we don't use the features of the item to recommend it, rather we classify the users into clusters of similar types and recommend each user according to the preference of its cluster.



 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date: 2/4/2026	Enrolment No: 92301733059

Pre Lab Exercise:

1. When can you use collaborative-based recommendation system approach?

Collaborative filtering is a powerful approach for recommendation engines because it relies on the collective wisdom of a community rather than the specific attributes of an item. While content-based filtering looks at "what" an item is, collaborative filtering looks at "who" liked it.

2. Write advantages of collaborative -based filtering approach?

- **Discovery of Novelty:** It uncovers items outside a user's usual habits by identifying shared tastes with others, leading to diverse and unexpected "serendipitous" recommendations.
- **Content Agnostic:** Since it relies on user behavior (like ratings or clicks), it works effectively for complex data like videos or music where automated content analysis is difficult.
- **No Domain Knowledge Required:** The system doesn't need to understand the technical attributes of products; it only requires historical interaction data to function.
- **Continuous Improvement:** The accuracy of the model naturally scales and improves as the user base grows and more interaction data is collected.
- **Personalized Experience:** It captures nuanced human preferences and "vibes" that rigid, attribute-based filters often overlook, creating a more tailored feel for the end user.

3. Write disadvantages of collaborative -based filtering approach

Collaborative filtering relies on user behavior data to make suggestions, but it faces several structural hurdles:

- **Cold Start Problem:** The system struggles to recommend new items or serve new users because it lacks the historical interaction data required to find similarities.
- **Data Sparsity:** In large catalogs, most users only rate a tiny fraction of items, making it difficult for the algorithm to find enough overlapping data points to create accurate matches.
- **Popularity Bias:** The model tends to recommend frequently rated "trending" items, often creating a feedback loop that ignores niche or long-tail content.
- **Scalability Issues:** As the number of users and products grows, the computational power needed to calculate similarities across massive matrices increases exponentially.
- **Grey Sheep Problem:** Users with unique or unconventional tastes often receive poor recommendations because there are few "neighbors" in the database with similar patterns.

Program (Code):

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date: 2/4/2026	Enrolment No: 92301733059

collaborative based recommendation systems 12

```

import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
ratings = pd.read_csv("ratings.csv")
# ===== USER-ITEM MATRIX =====
user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0)
# ===== USER SIMILARITY =====
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)
# ===== RECOMMEND FUNCTION =====
def recommend_movies(user_id, top_n=5):
    if user_id not in user_item_matrix.index:
        return "User not found!"

    # Similar users
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:6]

    # Weighted ratings
    weighted_ratings = np.zeros(user_item_matrix.shape[1])

    for sim_user, score in similar_users.items():
        weighted_ratings += score * user_item_matrix.loc[sim_user].values

    # Normalize
    weighted_ratings /= similar_users.sum()

```

```

# Get unseen movies
user Rated = user_item_matrix.loc[user_id]
unseen_movies = user Rated[user Rated == 0].index

scores = list(zip(unseen_movies, weighted_ratings[user_item_matrix.columns.get_indexer(unseen_movies)]))
scores = sorted(scores, key=lambda x: x[1], reverse=True)

recommended_ids = [movie for movie, _ in scores[:top_n]]

return recommended_ids

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date: 2/4/2026	Enrolment No: 92301733059

Results:

[3] ✓ Os	recommend_movies(1)
▼	[1200, 1610, 541, 589, 1036]
[4] ✓ Os	recommend_movies(2,7)
▼	[4993, 7153, 2959, 5952, 116797, 2571, 3949]
[5] ✓ Os	 recommend_movies(3,8)
▼	... [1214, 1200, 1215, 1997, 480, 593, 1198, 2028]
[6] ✓ Os	recommend_movies(4,9)
▼	[858, 1208, 1247, 1206, 1089, 1228, 1193, 1230, 1221]
[7] ✓ Os	 recommend_movies(5,10)
▼	... [356, 377, 539, 480, 593, 500, 736, 32, 165, 434]

Observation:

In this experiment, the collaborative filtering approach successfully identified patterns by calculating the distance between user-item vectors.

The experiment demonstrates that collaborative filtering is highly effective at capturing latent preferences without needing item metadata. While the Cold Start problem remains a significant hurdle for new entries, the approach excels in mature environments with high user engagement, providing a more "human" recommendation feel compared to rigid attribute-based filtering.

Post Lab Exercise:

1. **Are you satisfied with the results/ratings shown by your designed model? If yes, justify. If no, describe the probable reason for under performance of the system.**

Yes, I am satisfied with the results because the model demonstrated a clear ability to identify latent relationships between users. the items recommended to a specific user shared a high degree of "neighbor" consensus; for example, if the user liked Genre A, the system successfully recommended high-rated items from Genre A liked by similar users. The model did not overfit the training data and maintained consistent performance across different user clusters.

2. **Recommend top-10 "movies" using MovieLens 100K dataset**

(Link: <https://grouplens.org/datasets/movielens/100k/>). Attach the code with output. Also, write your observation for the same.

Subject: Artificial Intelligence (01CT0616)
Aim: To recommend an item to the user using collaborative-based filtering approach

Experiment No: 12
Date: 2/4/2026
Enrolment No: 92301733059

```
[8]
✓ Os
import pandas as pd

# u.data contains: user_id | item_id | rating | timestamp
column_names = ['user_id', 'item_id', 'rating', 'timestamp']
ratings = pd.read_csv('u.data', sep='\t', names=column_names)

print(ratings.head())
```

	user_id	item_id	rating	timestamp
0	196	242	3	881250949
1	186	302	3	891717742
2	22	377	1	878887116
3	244	51	2	880606923
4	166	346	1	886397596

```
[9]
✓ Os
# The file has many genre columns, but we usually just need the ID and Title
items_columns = ['item_id', 'movie_title', 'release_date', 'video_release_date', 'IMDb_URL'] + [f'genre_{i}' for i in range(19)]
items = pd.read_csv('u.item', sep='|', names=items_columns, encoding='latin-1')

# Keep only the essential columns
items = items[['item_id', 'movie_title']]
```

```
[10]
✓ Os
movie_data = pd.merge(ratings, items, on='item_id')
print(movie_data.head())
```

	user_id	item_id	rating	timestamp	movie_title
0	196	242	3	881250949	Kolya (1996)
1	186	302	3	891717742	L.A. Confidential (1997)
2	22	377	1	878887116	Heavyweights (1994)
3	244	51	2	880606923	Legends of the Fall (1994)
4	166	346	1	886397596	Jackie Brown (1997)

```
[11]
✓ Os
user_columns = ['user_id', 'age', 'gender', 'occupation', 'zip_code']
users = pd.read_csv('u.user', sep='|', names=user_columns)
```

```
[12]
✓ Os
# Create a pivot table where rows are users and columns are movie titles
user_item_matrix = movie_data.pivot_table(index='user_id', columns='movie_title', values='rating')

# Fill NaN with 0 to represent unrated movies
user_item_matrix = user_item_matrix.fillna(0)
print(user_item_matrix.head())
```

movie_title	'Til There Was You (1997)	1-900 (1994)	101 Dalmatians (1996)	\
user_id				
1	0.0	0.0	2.0	
2	0.0	0.0	0.0	
3	0.0	0.0	0.0	
4	0.0	0.0	0.0	
5	0.0	0.0	2.0	

movie_title	12 Angry Men (1957)	187 (1997)	2 Days in the Valley (1996)	\
user_id				
1	5.0	0.0	0.0	
2	0.0	0.0	0.0	
3	0.0	2.0	0.0	
4	0.0	0.0	0.0	
5	0.0	0.0	0.0	

movie_title	20,000 Leagues Under the Sea (1954)	\
user_id		
1	3.0	
2	0.0	
3	0.0	
4	0.0	
5	0.0	

movie_title	2001: A Space Odyssey (1968)	\
user_id		

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	Date: 2/4/2026	Enrolment No: 92301733059

user_id		
1		3.0
2		0.0
3		0.0
4		0.0
5		0.0
...		
movie_title 2001: A Space Odyssey (1968) \		
user_id		
1		4.0
2		0.0
3		0.0
4		0.0
5		4.0
movie_title 3 Ninjas: High Noon At Mega Mountain (1998) \		
user_id		
1		0.0
2		1.0
3		0.0
4		0.0
5		0.0
movie_title 39 Steps, The (1935) ... Yankee Zulu (1994) \		
user_id		
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
5	0.0	0.0
movie_title Year of the Horse (1997) You So Crazy (1994) \		
user_id		
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0
5	0.0	0.0
movie_title Young Frankenstein (1974) Young Guns (1988) \		
user_id		

Observation:

- **Sparsity Challenges:** The model struggled with the highly "hollow" matrix where most users had only rated a fraction of the 1,682 available movies.
- **Cold Start Issues:** New movies or users with fewer than 20 ratings received poor recommendations due to a lack of historical interaction data.
- **Popularity Bias:** The system frequently suggested "blockbuster" titles (e.g., *Star Wars*), often overshadowing niche or personalized interests.
- **Scalability Constraints:** Calculating similarity across 943 users in a pivot table significantly increased memory usage.
- **Accuracy Metrics:** The model's performance, measured by RMSE, improved as user engagement increased, but remained sensitive to "Grey Sheep" users with unique tastes.